AUTOMATA THEORY — CORE CONCEPTS

NFA vs DFA

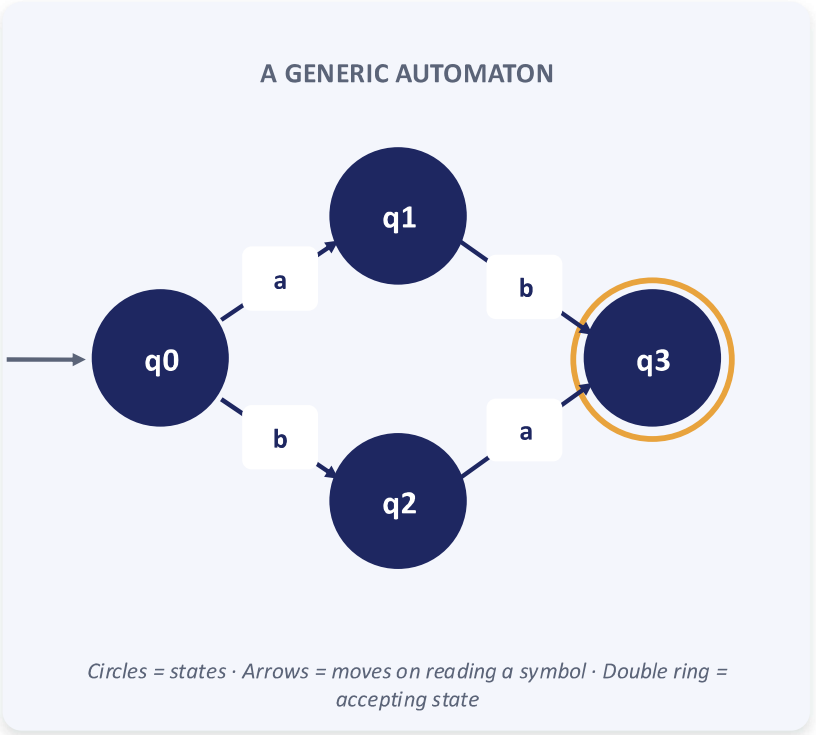
Two ways a machine can decide whether a string belongs to a language

What Is a Finite Automaton?

A finite automaton is a simple machine with a limited number of “states.” It reads a string one symbol at a time and jumps between states following fixed rules. If it ends in an accepting state, the string is recognized.

- Think of it like a board game with a few marked spots
- You move from spot to spot as you read each letter of a word
- Land on the “win” spot at the end — the machine accepts the word

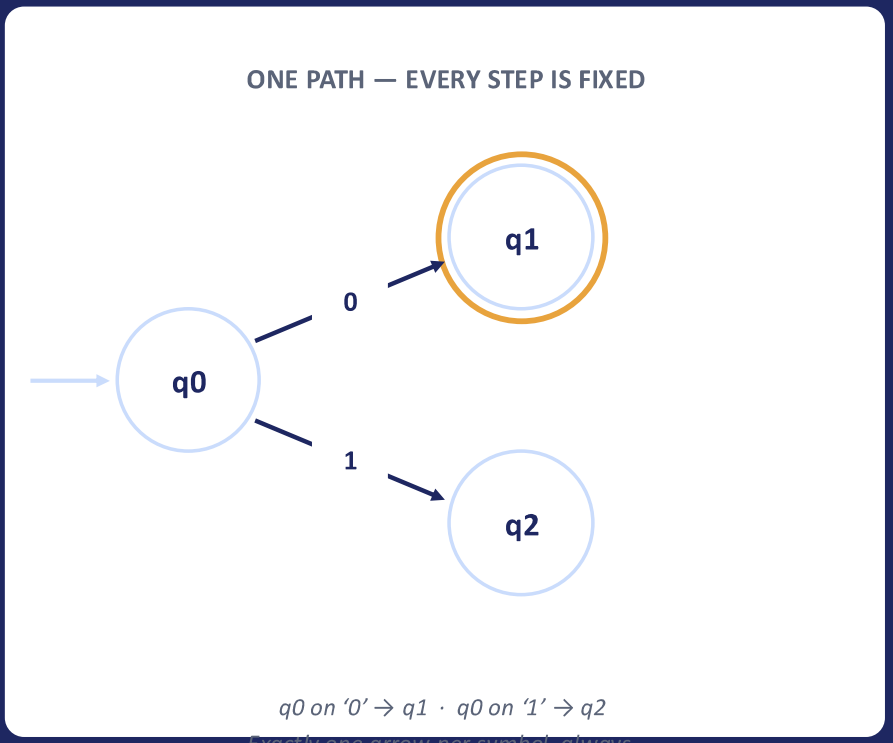
DFA and NFA are the two basic flavors of this machine — they differ only in how strict the movement rules are.



DFA

Deterministic Finite Automaton

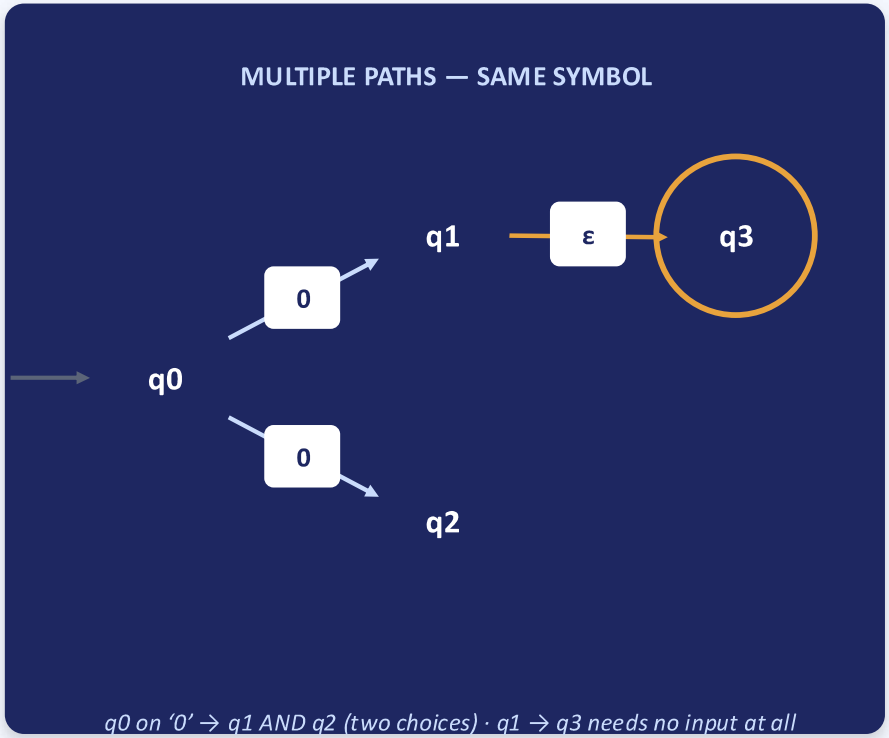
- For every state and every input symbol, there is exactly one next state
- Zero ambiguity — the machine always knows exactly where to go
- No “empty” (ε) moves — it must read a symbol to move
- Maps directly to code: a simple lookup table of (state, symbol) → state



NFA

Nondeterministic Finite Automaton

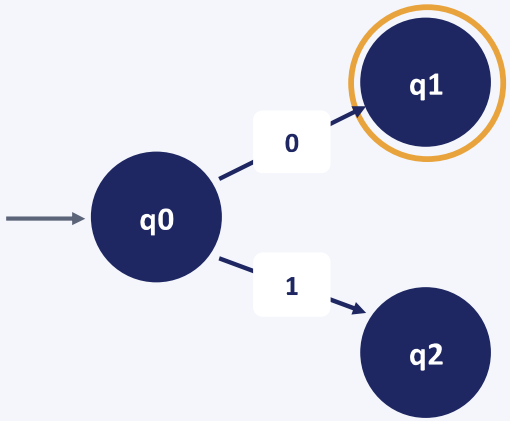
- A state can have zero, one, or many next states for the same symbol
- ε (epsilon) moves let it change state without reading any input
- The machine effectively explores several paths at once
- Often faster and more natural to design by hand than a DFA



Same Idea, Different Paths

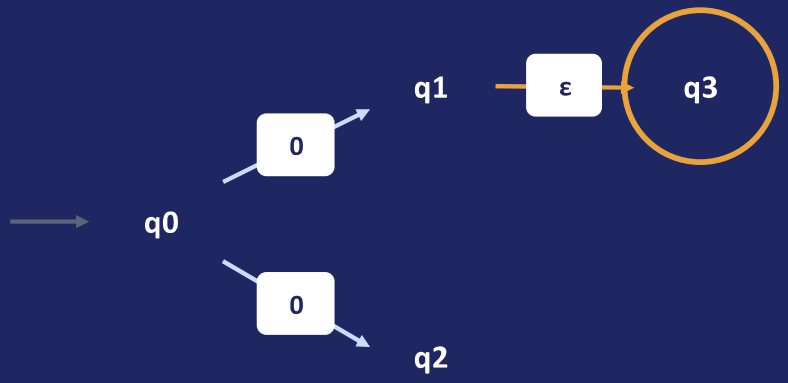
Placed side by side, the contrast becomes obvious at a glance

DFA



Exactly one arrow leaves q0 for each symbol — the next step is never in doubt.

NFA



Two arrows leave q0 on the very same symbol — both paths stay alive at once.

Key Differences at a Glance

Feature	DFA	NFA
Transitions per symbol	Exactly one	Zero, one, or many
Epsilon (ε) moves	Not allowed	Allowed
Ease of hand-design	Harder — must plan every case	Easier — allows guessing
States typically needed	Can be more	Often fewer
How it runs	Follows a single path	Conceptually explores many paths
Direct implementation	Simple lookup table	Needs simulation or conversion

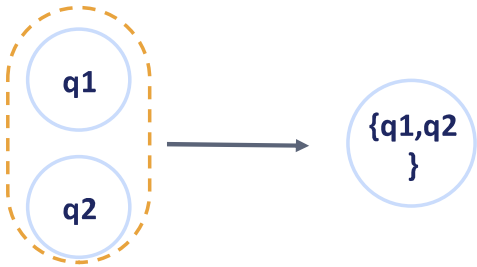
Are They Equally Powerful?

Yes — every NFA has an equivalent DFA

A process called the subset (powerset) construction turns any NFA into a DFA that accepts exactly the same strings. Both machine types recognize precisely the class of regular languages — neither can do something the other fundamentally cannot.

Design with NFA	Fewer states, natural “guessing,” faster to sketch by hand
Run with DFA	One path per input, predictable speed, ideal for execution
Real-world use	Regex engines often build an NFA, then convert it to a DFA

SUBSET CONSTRUCTION



Groups of NFA states become single DFA states

Choosing Between Them

Use a DFA when...

- You need fast, predictable execution
- The pattern is fixed and simple to trace
- You're implementing directly in code

Use an NFA when...

- You want an easy, natural design
- The pattern has many optional parts
- You'll convert it to a DFA before running it

Bottom line: DFA and NFA recognize the same languages — they only trade off design ease against execution simplicity.

Thank You